

Spring Boot – Introduzione

Definizione. Spring Boot è uno dei tanti progetti dell'ecosistema Spring (quando si dice "Spring" si intende un intero mondo di progetti, di cui Boot è oggi il più popolare). Il suo scopo è farti creare applicazioni Spring **riducendo al minimo la configurazione**: si occupa lui, dietro le quinte, di cose come la creazione del datasource, dei bean, ecc. Con Spring Boot puoi creare applicazioni Java standalone, web, ma soprattutto — ed è la novità più grossa — applicazioni con il **server incorporato (embedded)** dentro l'applicazione stessa. È anche uno strumento eccellente per progettare architetture a **microservizi**.

Il ribaltamento concettuale. Nel modello tradizionale hai un server e ci "deployi" dentro le applicazioni. Con Spring Boot è il contrario: è l'applicazione a contenere già il server (di solito **Tomcat**).

Come si crea un progetto. Il docente indica tre vie: installare Spring Boot, usare il sito ufficiale **Spring Initializr** (start.spring.io), oppure — la via più usata a lezione — direttamente da **Eclipse** con **File → New → Spring Starter Project**. In entrambi i casi scegli tipo di progetto (Maven), linguaggio, versione di Spring Boot, group/artifact, package e soprattutto le **dipendenze** (es. Spring Web, Spring Data JPA). Selezionando una dipendenza, Spring Boot si porta dietro in automatico tutte quelle collegate.

L'esempio della lezione. Crea un progetto "demo due" con Spring Web + Spring Data JPA. Nota che il `pom.xml` è già configurato e c'è già una classe `main` (quella con `@SpringBootApplication`) che avvia l'app. L'avvio si fa dalla vista **Boot Dashboard**. Al primo avvio va in eccezione: avendo aggiunto Spring Data, Boot vuole creare il datasource ma non sa a quale DB collegarsi, quindi bisogna aggiungere in `application.properties` url, username, password, dialetto SQL, più la dipendenza del **driver/connector** del database. Poi crea un semplice controller:

```
@RestController
@RequestMapping("/ctr")
public class HelloCtr {
    @RequestMapping("/hello")
    @ResponseBody
    public String hello() {
        System.out.println("dentro");
        return "Ciao";
    }
}
```

Infine mostra **spring-boot-devtools**: una dipendenza che ricarica le modifiche "a caldo" senza dover riavviare a mano il server ad ogni salvataggio.

File di Properties e YAML

Definizione. I file di properties servono a dichiarare le proprietà (configurazioni) di un'applicazione nel formato `nome=valore`. In Spring Boot il file di default è `src/main/resources/application.properties`. I due vantaggi principali: (1) tutte le proprietà sono raccolte in un unico file di testo; (2) puoi modificarle senza dover ricompilare/riavviare, perché il file è esterno alla logica.

Property personali vs property predefinite. Puoi definire property tue (es. `messaggio=Ciao mondo`), ma esistono anche **property già pronte** che Spring Boot riconosce, per esempio:

- `server.port=8080` → porta su cui parte l'app
- `spring.application.name=...` → nome dell'applicazione

L'elenco completo è nella documentazione ufficiale (sezione *application properties*).

Leggere una property dal codice con `@Value`, con possibilità di valore di default dopo i due punti:

```
@Value("${messaggio:benvenuto}")
```

```
private String tmp;
```

Se la property non esiste e non hai messo il default, Spring Boot va in eccezione all'avvio.

YAML. Spring Boot supporta anche `application.yml`, formato alternativo (più leggibile) per le stesse informazioni. Esempio equivalente:

```
server:
```

```
  port: 8080
```

```
spring:
```

```
  application:
```

```
    name: la-mia-app
```

Profili. Puoi avere file di property distinti per ambiente: `application-test.properties`, `application-prod.properties`, ecc. Per scegliere quale attivare, lo specifichi all'avvio da riga di comando:

```
java -jar HelloWorldSpringBoot.jar --spring.profiles.active=test
```

Con YAML puoi invece tenere tutto in un unico file separando i blocchi dei profili con `--`.

Spring Boot – Gestione Eccezioni

Definizione. Si tratta di centralizzare la gestione degli errori in modo che, quando un servizio REST lancia un'eccezione, al client torni una risposta pulita e ben formata invece di uno stack trace. Il meccanismo si basa su una classe annotata con `@RestControllerAdvice` (nella trascrizione: `@RestController + @ControllerAdvice`) che **estende**

`ResponseEntityExceptionHandler`, con un metodo per ogni tipo di eccezione, annotato con `@ExceptionHandler`.

L'esempio della lezione. Crea due eccezioni custom:

- `NotFoundException` → lanciata quando una ricerca per id non trova nulla
- `DuplicateException` → lanciata quando ci si aspetta un solo risultato ma ne tornano più di uno

Ogni eccezione estende `Exception`, ha un `serialVersionUID`, un campo `messaggio`, due costruttori (con e senza argomenti) e getter/setter. Poi crea una classe `ErrorResponse` con i campi `codice` e `messaggio`, e la classe handler:

`@RestControllerAdvice`

```
public class RestExceptionHandler extends ResponseEntityExceptionHandler {
```

```
    @ExceptionHandler(DuplicateException.class)
```

```
    public ErrorResponse handleDuplicate(DuplicateException ex) {
```

```
        ErrorResponse err = new ErrorResponse();
```

```
        err.setCodice(HttpStatus.CONFLICT.value()); // 409
```

```
        err.setMessaggio(ex.getMessage());
```

```
        return err;
```

```
    }
```

```
    @ExceptionHandler(NotFoundException.class)
```

```
    public ErrorResponse handleNotFound(NotFoundException ex) {
```

```
        ErrorResponse err = new ErrorResponse();
```

```
        err.setCodice(HttpStatus.NOT_FOUND.value()); // 404
```

```
        err.setMessaggio(ex.getMessage());
```

```
        return err;
```

```
    }
```

```
}
```

Nota importante sugli HTTP status. Nei servizi REST c'è la convenzione di riutilizzare i codici HTTP come codici di risposta. La tabella data a lezione: **200** OK · **201** creato (insert) · **204** cancellazione · **304** non modificato · **400** richiesta non valida · **401/403** autorizzazione (password sbagliata / permessi mancanti) · **404** not found · **409** risorsa duplicata · **422** dati mancanti/non processabili · **500** errore generico non previsto.

Il controller di test lancia sempre `DuplicateException`; chiamando l'endpoint, l'handler intercetta e restituisce l'`ErrorResponse` con codice 409.

Spring Boot – Logging

Definizione. Il logging è la registrazione controllata di ciò che accade nell'applicazione. Rispetto a `System.out.println`, un framework di logging permette di controllare **livello**, **formato**,

dimensione dei file, rotazione/backup. Spring Boot ha il logging integrato: basta configurarlo in `application.properties` o `application.yml`.

Livelli e priorità. Dal più verboso al meno: **TRACE** → **DEBUG** → **INFO** → **WARN** → **ERROR** → **FATAL**. Impostando il livello a **INFO**, vengono stampati **INFO** e tutto ciò che è più grave (**WARN**, **ERROR**, **FATAL**); **DEBUG** e **TRACE** restano nascosti. Regola pratica del corso: in **produzione** si usa **INFO**, in **sviluppo** **DEBUG** (**TRACE** stampa tutto ma rallenta l'avvio).

Esempio di configurazione YAML (dalla lezione):

```
server:
  port: 8082
spring:
  application:
    name: demo3
logging:
  file:
    name: demo3.log
  logback:
    rollingpolicy:
      max-file-size: 1MB
  level:
    root: info
    org.hibernate: debug
    org.springframework.web: info
```

Uso nel codice:

```
private static final Logger logger = LoggerFactory.getLogger>HelloWorldController.class);
```

```
@GetMapping("/hello")
public String hello() {
    System.out.println("...");
    logger.info("messaggio info");
    logger.debug("messaggio debug");
    return "Ciao mamma";
}
```

Con livello `info`, la `logger.debug(...)` non compare; alzando a `debug` compaiono entrambe. Il docente fa notare quante più informazioni fornisce il log rispetto al semplice `System.out` (data, livello, thread, PID, classe che ha stampato).

Spring Data JPA con Spring Boot

Definizione. Spring Data JPA è il modulo che semplifica l'accesso al database tramite **repository**: definisci un'interfaccia che estende `JpaRepository` e ottieni gratis i metodi CRUD, più la possibilità di definire query "derivate" dal nome del metodo o scritte a mano. La lezione mostra **come configurarlo dentro Spring Boot** (dando per scontato che si conosca già Spring Data JPA in sé).

Passi di configurazione:

1. Dipendenza Maven `spring-boot-starter-data-jpa` (selezionata come "Spring Data JPA" nello starter).
2. Connettore/driver del DB (es. MySQL connector).
3. In `application.properties`, le property essenziali:

```
spring.datasource.url=jdbc:mysql://localhost:3306/prova
spring.datasource.username=...
spring.datasource.password=...
spring.jpa.show-sql=true
```

Esempio. Ci si collega a un DB prova con tabella `country` (mappata da una entity `Country`). Il repository:

```
public interface CountryRepository extends JpaRepository<Country, Integer> {
    Country findByCountryName(String name);    // query derivata
    // + un metodo con @Query SQL nativa
    // + un metodo con @Query JPQL
    // + cancellazione per nome
}
```

Sopra c'è un `@RestController` che espone questi metodi (ricerca per nome, elenco di tutte le righe, ricerca "like", cancellazione). Chiamando `localhost:8080/api/all` torna tutta la tabella.

Esercizio Svolto n1

(NetFliz.zip/MagazzinoSpringBoot)

Traccia. Creare con Spring Boot un'app che gestisce un **articolo** (codice, descrizione, prezzo): database, Spring Data con repository e un controller con un metodo che restituisce tutti gli articoli.

Svolgimento. Crea il DB magazzino_spring_boot con tabella articolo (codice intero auto-increment come chiave primaria, descrizione VARCHAR(500), prezzo). Progetto Spring Boot con Spring Web, Spring Data, DevTools e **Lombok**. Struttura a package: model, repository, service, ctr.

- **Entity** Articolo con @Entity, attributi (codice, descrizione, prezzo) e Lombok per getter/setter/costruttore.
- **Repository:** interface ArticoloRepository extends JpaRepository<Articolo, Integer>.
- **Service** con @Service, repository iniettato via @Autowired, metodo getArticles() che ritorna articoloR.findAll().
- **Controller** (dal codice reale):

```
@RestController
@RequestMapping("/articolo")
public class ArticoloCtr {
    @Autowired
    private ArticoloService articoloS;

    @GetMapping
    public List<Articolo> getArticles(){
        return articoloS.getArticles();
    }
}
```

Ricorda di aggiungere il driver nel pom.xml (dettaglio dimenticato e poi corretto a lezione).
Chiamando localhost:8080/articolo/ con un record inserito, torna l'articolo.

Esercizio Svolto n.2

(complemento Magazzino)

Obiettivo. Aggiungere all'esercizio precedente i metodi mancanti: **inserimento, update, cancellazione** (POST, PUT, DELETE).

- **Insert** — @PostMapping + @RequestBody per ricevere l'articolo dal corpo della richiesta; il service chiama articoloR.save(a) e il controller torna ResponseEntity<String> con "inserimento effettuato" e HTTP status. Testato via **Postman** con JSON { "descrizione": "Coca Cola", "prezzo": 2 } (il codice lo genera il DB).

- **Update** — quasi identico ma `@PutMapping`; usa sempre `save`, che fa insert o update a seconda che la chiave esista. Attenzione: nel JSON va passato il campo chiave con il nome corretto (`codice`, non `id`).
- **Delete** — `@DeleteMapping`; il service chiama `deleteById(a.getCodice())`.

Morale della lezione: `save()` copre insert e update, il resto è quasi copia-incolla cambiando l'annotation del verbo HTTP.

Caratteristiche degli Ambienti

Sviluppo – Test – Pre-Produzione – Produzione

Definizione. È il "percorso" che un'applicazione compie prima di arrivare agli utenti reali. In un'azienda medio-grande ci sono in genere **quattro ambienti**:

1. **Sviluppo** — il programmatore lavora prima sulla sua macchina, poi installa nell'ambiente di sviluppo (di solito una macchina Linux con server e servizi). I servizi esterni spesso sono **simulati** (es. un finto PayPal che risponde sempre "ok"). Ci lavorano tutti gli sviluppatori.
2. **Test** — un gruppo dedicato di tester ha una propria macchina; esegue test (regressione, ecc.). **I dati sono fittizi.**
3. **Pre-produzione** — ambiente molto simile al reale, con **dati equivalenti a quelli di produzione**; si ripetono i test. Se qualcosa non va, l'app torna indietro con un ticket allo sviluppatore.
4. **Produzione** — l'ambiente reale, ciò che vede il cliente (es. il sito vero). Qualcuno "ci mette la faccia" approvando il passaggio.

Gravità dei bug crescente: un bug in test è banale (si rimanda indietro), in pre-produzione è più serio, in **produzione** è critico — soprattutto se l'app tratta soldi — e ha priorità alta. Nonostante tutti i controlli, capita che qualche bug arrivi comunque in produzione.

I Profili con Spring Boot

Definizione. Un profilo è un **insieme di configurazioni valido solo in un certo ambiente**. Poiché DB, indirizzi di servizi esterni, messaggi e perfino la porta cambiano tra sviluppo, test e produzione, si creano più file di property, uno per profilo.

Come si fa.

- `application.properties` → configurazione valida sempre
- `application-test.properties` → ambiente di test
- `application-prod.properties` → produzione (cambia ad es. `server.port`)

Attivazione da riga di comando:

```
java -jar HelloWorldSpringBoot-0.0.1-SNAPSHOT.jar --spring.profiles.active=test
```

Con prod avvii il profilo di produzione; senza il suffisso usi lo sviluppo (`application.properties`). È lo stesso meccanismo già anticipato nella lezione 2, qui inquadrato negli ambienti della lezione 8.

Actuator

(HelloWorldSpringBootActuator.zip)

10. Actuator (HelloWorldSpringBootActuator.zip)

Definizione. Spring Boot Actuator fornisce una serie di **endpoint** per **monitorare e gestire** i microservizi (memoria, salute, metriche, mapping, ecc.). Necessario proprio perché, con tanti microservizi al posto di un'unica app monolitica, servono strumenti di controllo puntuali.

Configurazione.

1. Dipendenza `spring-boot-starter-actuator`.
2. In `application.properties` scegli quali endpoint esporre (a lezione: esporli tutti con `management.endpoints.web.exposure.include=*`).

Endpoint mostrati (via Postman):

- radice actuator → elenco di tutti gli endpoint disponibili
- `/actuator/health` → se il microservizio è su/giù (UP)
- `/actuator/metrics` → metriche; poi si scende nel dettaglio, es. CPU usata, richieste HTTP gestite
- `/actuator/mappings` → come sono mappati i microservizi (GET, DELETE, ecc.)
- `/actuator/info` → informazioni personalizzabili

Info statiche (fisse, definite nel file di property, es. `info.assistenza.nome=Pippo`, email, telefono).

Info dinamiche (es. da query): implementi `InfoContributor`. Dal codice reale del progetto:

```
@Component
public class ArtInfoContributor implements InfoContributor {

    @Override
    public void contribute(Builder builder) {
        Map<String, Object> artMap = new HashMap<>();
        artMap.put("giorni liberi", 2);
        builder.withDetail("microservizio-info", artMap);
    }
}
```

Info sulla build — aggiungendo l'apposito plugin Maven nel `pom.xml`, l'endpoint `info` mostra anche versione, artifact, group dell'applicazione.

Pattern DTO

Definizione. **DTO = Data Transfer Object.** È il pattern per cui **non si fa "viaggiare" il model (l'entity) verso l'esterno**, ma una classe apposita creata solo per il trasferimento dei dati. Perché il model non deve uscire: (1) esporlo rivela la struttura del database; (2) presto le entity JPA avranno annotazioni e campi interni non necessari — anzi indesiderati — al client.

Meccanica. Se avevi `Corso getId(int id)`, con il DTO diventa `CorsoDto getId(int id)`. Crei un package `dto` con una classe `CorsoDto` (all'inizio quasi un copia-incolla dei campi di `Corso`, ma destinata a divergere). Nei metodi devi **convertire** `model → DTO` (in uscita) e `DTO → model` (in ingresso, es. in un `aggiornaCorso`).

Il Builder. Per non riscrivere le conversioni in ogni metodo, si crea una classe di supporto (es. `CorsoBuilder`) con due metodi `public static`:

```
public class CorsoBuilder {  
    public static CorsoDto fromModelToDto(Corso c) { /* copia i campi */ }  
    public static Corso fromDtoToModel(CorsoDto d) { /* copia i campi */ }  
}
```

Il docente ammette che farlo a mano è noioso — ed è proprio il problema che le due lezioni successive risolvono con librerie automatiche.

Applicazione del Pattern DTO a un servizio REST tramite ModelMapper

(DtoMapperSpringBootRest.zip)

Definizione. **ModelMapper** è una libreria che converte **automaticamente** un'entity nel suo DTO (e viceversa), evitando i `get/set` manuali campo per campo — cruciale quando una classe ha decine di attributi.

Configurazione.

1. Dipendenza `modelmapper`.
2. Nella classe di avvio, definisci un bean:

```
@Bean  
public ModelMapper modelMapper() {  
    return new ModelMapper();  
}
```

```
}
```

3. Nel controller/service, inietti con `@Autowired ModelMapper modelMapper;`.

Uso. Il metodo chiave è `map(istanzaDaTrasformare, ClasseDestinazione.class)`:

```
private StudenteDto convertToDto(Studente s) {  
    return modelMapper.map(s, StudenteDto.class);  
}  
  
private Studente convertToEntity(StudenteDto sDto) {  
    return modelMapper.map(sDto, Studente.class);  
}
```

Per una **lista** basta scorrere e mappare ogni elemento (con `for` oppure con uno `stream` `.map(this::convertToDto)`). Nel progetto `Studente` ha `matricola/nome/cognome` e `StudenteDto` le stesse info. Chiamando `findById`, torna un `StudenteDto` invece dell'entity.

MapStruct

(springbootmodelmapper.zip)

Definizione. **MapStruct** è un'altra libreria per mappare `DTO ↔ entity`, ma genera il codice di mapping **a compile-time** (senza reflection).

Configurazione.

1. Dipendenza `org.mapstruct:mapstruct` (+ il processor).
2. Un'interfaccia per ogni entity, annotata con `@Mapper`. Dal codice reale:

`@Mapper`

```
public interface AutoStudenteMapper {  
    AutoStudenteMapper MAPPER = Mappers.getMapper(AutoStudenteMapper.class);  
  
    StudenteDto mapToStudenteDto(Studente s);  
    Studente mapToStudente(StudenteDto sDto);  
}
```

Basta dichiarare le **firme** dei metodi: **MapStruct** genera l'implementazione. Nel service lo usi così: `AutoStudenteMapper.MAPPER.mapToStudenteDto(s)`.

Confronto ModelMapper vs MapStruct (la parte più utile della lezione):

- Con il **mapper manuale** (StudenteMapper nel progetto) scrivi tu i get/set — verboso.
- **ModelMapper**: più facile e intuitivo da configurare (basta il bean + `map( . . . )`), ma **più lento** perché usa la reflection.
- **MapStruct**: più veloce, ma la configurazione iniziale è più laboriosa (interfaccia, annotazioni, metodi).

In sintesi: *ModelMapper* = comodità, *MapStruct* = prestazioni; si sceglie caso per caso.

Validazione dei servizi REST

(SpringBootRestValidation.zip)

Definizione. Validare i dati in ingresso di un servizio REST significa verificare che il JSON ricevuto rispetti certi vincoli, prima di elaborarlo. Si usa `spring-boot-starter-validation` (che porta con sé **Hibernate Validator**), le annotazioni di vincolo sui campi (es. `@NotBlank`) e `@Valid` sul parametro del controller.

Esempio reale. La classe User:

```
public class User {  
    private long id;  
  
    @NotBlank(message = "Name is mandatory")  
    private String name;  
  
    @NotBlank(message = "Email is mandatory")  
    private String email;  
    // getter/setter  
}
```

Il controller con validazione e gestione dell'errore:

```
@RestController  
public class HelloController {  
  
    @PostMapping("/users")  
    ResponseEntity<String> addUser(@Valid @RequestBody User user) {  
        return ResponseEntity.ok("User valido");  
    }  
}
```

```

@ResponseStatus(HttpStatus.BAD_REQUEST)
@ExceptionHandler(MethodArgumentNotValidException.class)
public Map<String, String> handleValidationExceptions(MethodArgumentNotValidException
ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getAllErrors().forEach(error -> {
        String fieldName = ((FieldError) error).getField();
        String errorMessage = error.getDefaultMessage();
        errors.put(fieldName, errorMessage);
    });
    return errors;
}
}

```

Punti chiave della lezione. Senza `@Valid`, le annotazioni `@NotBlank` **non scattano**. Con `@Valid`, un campo mancante fa lanciare `MethodArgumentNotValidException` (di default una risposta 400 grezza). Il metodo `@ExceptionHandler` intercetta l'eccezione e restituisce una **mappa { campo: messaggio }** leggibile (es. email: "Email is mandatory"), con `@ResponseStatus(HttpStatus.BAD_REQUEST)`. Esistono molte altre annotazioni di vincolo nella documentazione ufficiale.

Test con Spring Boot

Definizione. Panoramica introduttiva sui test di un `@RestController`. Serve la dipendenza `spring-boot-starter-test`. Sulla classe di test si mettono `@SpringBootTest` e `@AutoConfigureMockMvc` (che permette di testare i controller **senza avviare un server reale**), e si inietta `MockMvc` con `@Autowired`.

Struttura. Buona norma: una classe di test per ogni controller. Ogni metodo di test ha `@Test`. Si invoca l'endpoint con `mockMvc.perform(get(...))`, si stampa la risposta con `.andDo(print())` e si dichiarano le attese con `.andExpect(...)`.

Esempi della lezione. L'app ha un controller con due metodi GET: uno che ritorna la stringa "Hello2" su `/api/esempio`, uno che ritorna un JSON Country (id, countryName, population) su `/api/cerca/{id}`.

`@SpringBootTest`

`@AutoConfigureMockMvc`

```

class HelloWorldSpringTestApplicationTests {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void apiEsempioTest() throws Exception {
        mockMvc.perform(get("/api/esempio"))
            .andExpect(print())
            .andExpect(content().string("Hello2"));
    }

    @Test
    void ricercaPerIdTest() throws Exception {
        mockMvc.perform(get("/api/cerca/1"))
            .andExpect(print())
            .andExpect(jsonPath("$.countryName").value("Italia"));
    }
}

```

Concetti chiave. Per una risposta stringa si verifica con `content().string(...)`; per un JSON si usa `jsonPath("$.proprietà").value(...)`, che permette di navigare il JSON. Test verde = riuscito. È ribadito che è solo un'introduzione basica.

Le Transazioni in Spring Boot – Livelli di Propagazione

Definizione. Una transazione si crea annotando un metodo del service con `@Transactional`. Il **livello di propagazione** definisce cosa succede quando un metodo transazionale ne chiama un altro: se riusare la transazione esistente, crearne una nuova, sospenderla, o lanciare eccezione. Scenario di lezione: `ServiceEsempio.hello()` (chiamante) invoca `CountryService.create()` (chiamato).

```

@Service
public class ServiceEsempio {

    @Autowired private CountryService cs;

```

```
@Transactional

public void hello(Country c) {
    cs.create(c);
}
}
```

I livelli (dai commenti nel codice reale CountryService):

- **REQUIRED** (*default*) — se il chiamante ha già una transazione la usa, altrimenti ne crea una nuova. `@Transactional` senza parametri equivale a questo.
- **REQUIRES_NEW** — crea **sempre** una nuova transazione, sospendendo l'eventuale transazione chiamante.
- **SUPPORTS** — se esiste una transazione chiamante la usa, altrimenti il metodo gira **senza** transazione (adattivo).
- **NOT_SUPPORTED** — il metodo gira **sempre senza** transazione; se ne esiste una la sospende.
- **MANDATORY** — deve girare **dentro** una transazione; se non esiste, **lancia eccezione**.
- **NEVER** — deve girare **sempre senza** transazione; se ne esiste una, **lancia eccezione** (l'opposto di `NOT_SUPPORTED`).
- **NESTED** — se esiste una transazione crea un **punto di ripristino (checkpoint)**: se il codice va in eccezione fa **rollback** fino a quel punto; se non esiste transazione, si comporta come `REQUIRED`.

Sintassi: `@Transactional(propagation = Propagation.REQUIRES_NEW)`.

Le Transazioni in Spring Boot – Livelli di Isolamento

Definizione. Il **livello di isolamento** definisce come la transazione si comporta rispetto ai dati quando ci sono **transazioni concorrenti** sugli stessi dati. Sintassi:

`@Transactional(isolation = Isolation.SERIALIZABLE)`. C'è un compromesso costante tra **sicurezza dei dati** e **prestazioni**.

I livelli (dal più sicuro/lento al più veloce/rischioso, secondo i commenti del codice):

- **DEFAULT** — usa il livello di isolamento del **database sottostante**.
- **SERIALIZABLE** — isolamento massimo: agisci come se fossi l'unica transazione su quei dati. Nessuna anomalia, ma se al commit i dati sono stati modificati da altri vai in **rollback** → molti rollback possibili → **problemi di performance**.
- **REPEATABLE_READ** — i dati letti vengono **bloccati**: nessun altro può modificarli. Più veloce di `SERIALIZABLE`, ma blocca molte righe e soffre delle **letture fantasma** (ri-eseguendo la stessa query puoi trovare **più righe** aggiunte nel frattempo; quelle che hai bloccato però non cambiano).
- **READ_COMMITTED** — niente lock in lettura (più performante), ma compaiono anche le **letture non ripetibili** (ri-eseguendo la stessa query nella transazione puoi ottenere dati diversi, se un'altra transazione li ha modificati e committati).

- **READ_UNCOMMITTED** — il più veloce, ma oltre alle anomalie precedenti aggiunge le **letture sporche** (dirty read): puoi leggere dati modificati da un'altra transazione che **non ha ancora fatto commit**, quindi rischi dati inconsistenti. (Nel progetto il metodo *create* usa proprio **READ_UNCOMMITTED** come esempio attivo.)

Le tre anomalie in ordine di gravità crescente: **letture fantasma** → **letture non ripetibili** → **letture sporche**. In pratica si può **combinare** propagazione e isolamento, es.

`@Transactional(propagation = Propagation.REQUIRES_NEW, isolation = Isolation.SERIALIZABLE)`. Presupposti dati per noti: cosa sono transazione, commit e rollback.

Spring vs Spring MVC vs Spring Boot

Definizione / confronto. Tre livelli complementari, non concorrenti:

- **Spring (core)** — incentrato sulla **Dependency Injection**: definire le dipendenze (con `@Component`, `@Service`, ecc.), identificarle tramite il **component scan**, collegarle con `@Autowired`. Da solo però non basta per app grandi: servono altri moduli.
- **Spring MVC** — modulo che semplifica la creazione di **web application e API REST**. Prima esisteva Struts, considerato troppo complesso; Spring MVC lo ha semplificato. Fornisce `@Controller`, `@RestController`, `@RequestMapping`, ecc. Richiede però ancora un po' di **configurazione** (pom.xml, eventuale web.xml...).
- **Spring Boot** — **non** è un concorrente ma una specie di **wrapper** sopra Spring/Spring MVC: costruisci rapidamente applicazioni pronte per il deploy, con **server embedded**, e con gestione facilitata di logging, errori, profili e configurazioni. Riduce la configurazione all'osso.

In una frase: *Spring* mette la DI, *Spring MVC* rende facili le web app, *Spring Boot* avvolge tutto togliendo la configurazione.

Server e Embedded Server

Definizione. La lezione contrappone i due modelli di deployment.

Modo tradizionale (server esterno). Hai un server (es. **Tomcat**) su cui deployi più applicazioni, tipicamente impacchettate come file **.war**. Esempio: costruisci con Maven `prova-maven-0.0.1-SNAPSHOT.war`; in produzione il Tomcat è su un'altra macchina (di solito Linux) e tu **copi il .war nella cartella webapps** e avvii il server. Il server è **esterno** e ospita più applicazioni.

Server embedded. Con i microservizi ogni applicazione è un **.jar** che **contiene già il suo server** al proprio interno. Non deployi l'app dentro un server: è il server a stare dentro l'app. Basta:

```
java -jar HelloWorldSpringBoot.jar
```

e parte una versione di Tomcat semplificata (più veloce). Questo modello — **un server embedded per ciascuna applicazione** — è quello tipicamente usato nelle **architetture a microservizi**, ed è la grande novità di Spring Boot già annunciata nella prima lezione.